Sterl: Banking App Plan for JPMorgan

Nov 2, 2026 · updated Nov 4, 2026 · @Mehak Javed

What JPMorgan wants from a React developer

JPMorgan's London React roles are mostly in the Commercial & Investment Bank, building trading and markets screens. They ask for React + TypeScript, testing, data grids and fast real-time updates.

Skill	How JPMorgan's London postings phrase it	Your sprint week
React + TypeScript	Required in every posting reviewed; "production web applications with React and TypeScript"	Already have; deepen in weeks 1–2
Testing	Jest and React Testing Library listed as required	Week 3
State management + async data	Redux and hooks; "state management and asynchronous data flows"	Week 1
Real-time, high-frequency data	Live trading screens with "high-frequency updates"	Week 3 (performance)
Data grids	AG Grid or similar advanced grids, preferred	Week 3
Performance	Profiling, tuning, browser performance	Week 3
Security	Secure code and awareness of web security risks	Weeks 2–3, then backend phase
CI/CD + observability	CI/CD pipelines, logging, metrics, alerting, incident triage	Week 3, then Phase 4
AI coding tools	Using approved AI assistants and checking their output for correctness and security	Every week
Finance knowledge	Trading, orders, FX or futures & options exposure, preferred	Learn while building Sterl
A second language	"One or more extra programming languages", preferred; Java is common at banks	Backend phase

Experience reality check: the posted London React roles ask for 3+ years. Without that, your entry route is JPMorgan's graduate Software Engineer Program, internships or the Code for Good hackathon. Sterl is your proof for those routes, and for other UK banks and fintechs too.

Why Sterl is a website, not a phone app NEW

Sterl is a **responsive web app**: a website that looks and works like a banking app on a phone, and like a full trading desk on a laptop.

Reason	What it means for you
It matches the job	JPMorgan's London React roles build browser-based screens. A web app proves the exact skills they test; a native phone app (Swift, Kotlin, React Native) proves different ones.
Recruiters can open it in seconds	A live link on your CV opens instantly. Nobody has to install anything.
It fits a one-month launch	Deploying to Vercel takes minutes. App store reviews, signing and device testing would use up your month.
Mobile is still covered	A mobile-first layout makes Sterl feel like an app on a phone. Later you can make it installable to the home screen as a PWA.

How a bank sees software

A bank cares less about flashy features and more about trust: money must never be wrong, lost or stolen. Showing you think this way sets you apart from other candidates.

Bank concern	What it means for your code	How Sterl shows it
Money accuracy	Never store money as decimals; use integer pence. Balances must always add up	All amounts stored in pence; tests prove round-ups and totals are exact
Security	Protect logins, sessions and data; stop XSS, CSRF and injection	Re-authentication before sensitive actions; card details hidden until confirmed
Auditability	Regulators need a record of who did what, and when	An audit log of every transfer, card freeze and setting change
Resiliency	The app must cope when a service is slow or down	Clear loading, error and retry states; a payment can't be sent twice
Real-time performance	Trading screens update many times a second without freezing	Live FX panel streaming thousands of simulated price updates smoothly
Observability	Find and fix problems in production fast	Error tracking and logging hooked up from day one
Accessibility	UK banks must serve every customer, including disabled users	Keyboard-only and screen-reader tested; high-contrast mode

Bank concern	What it means for your code	How Sterl shows it
Responsible AI	Use AI tools, but verify their output	README section on how AI-written code was reviewed and tested
Data protection NEW	Never handle real customer data without a legal reason	Every person, account and payment in Sterl is generated fake data

In interviews, mention these trade-offs when you walk through your project. For example: "I used idempotency keys so a double-click can't send a payment twice."

Sterl: the web app

Sterl is a banking web app with two sides: a personal banking dashboard inspired by Chase UK, and a live markets desk inspired by JPMorgan's trading screens. Together they show both the customer-facing skills and the real-time, data-heavy skills JPMorgan hires for.

Use your own brand, colours and logo, never Chase's or JPMorgan's. Use fake data only (see "Data: 100% fake, no approval needed" below).

What Chase UK does (the reference)

The Chase UK app offers round-ups into a savings account, spending insights with budgets, multiple accounts for budgeting, a numberless card with details in the app, and fee-free spending abroad. Sterl covers these basics, then adds the features below.

Core features

Feature	What it does	Skills it proves
Accounts dashboard	Balances across current, saver and round-up accounts	Server Components, TanStack Query
Transactions	Searchable, filterable list with categories	Data grid, filtering, pagination
Send money	Pay a saved or new (fake) payee with a review step; only moves fake money between fake accounts	React Hook Form + Zod, server actions
Cards	Freeze/unfreeze; reveal card details after re-auth. Card numbers are published test numbers, never real ones	Zustand UI state, security patterns
Round-ups	Round each payment up to the nearest £1 into savings	Exact money maths, unit tests

Innovative features (what makes it stand out)

Feature	The idea	Why JPMorgan would care
Live FX desk	A grid of 30+ currency pairs ticking several times a second, with a price chart and a mock "order" button. Prices start from free daily reference rates, then a simulator makes them tick	Matches their trading UI roles: real-time data, AG Grid, performance
Payee name check	Before paying, warns if the name doesn't match the account. Simulated against SterI's own fake payee list, the same idea as UK Confirmation of Payee (the real service is only for regulated payment firms)	Fraud prevention, a real UK banking rule
Fraud alert simulator	Flags unusual payments (new country, large amount) and asks "Was this you?"	Risk thinking, event-driven UI
Smart round-up rules	Multiply round-ups, or pause them when balance is low	State logic, testable business rules
Subscription detector	Finds repeat payments and shows the yearly cost	Data analysis on transactions
Audit trail page	A timeline of every sensitive action	Auditability, which regulators require
Ask SterI (AI)	"How much did I spend on food in October?" answered from your fake transactions	Responsible AI integration
Accessibility mode	High contrast, larger text, full keyboard use	Inclusive design for every customer

Data: 100% fake, no approval needed NEW

Yes: you can build all of SterI without FCA authorisation or any other third-party approval. FCA rules apply when you hold real customers' money, read their real bank accounts, or start real payments. SterI does none of those. It is a **demo bank** that runs on data you generate yourself, plus public exchange rates.

> **What changed from the first version:** the payee name check is now simulated against fake payees, card numbers come from published test numbers, FX prices are simulated on top of free public reference rates, the AI feature only ever reads fake data, and the "make it a real bank" phase is now "make it work like a real bank" (still with fake money).

Where each kind of data comes from

Data Sterl needs	Source	Approval or sign-up needed?
Customers, accounts, balances, payees, 500+ transactions	Your own generator using **Faker.js** (`@faker-js/faker`), with a fixed seed so the data is the same every run	None. It's a free npm package
The "bank API" your screens call	Your own **Next.js route handlers** (`app/api/.../route.ts`) serving the generated data	None. You write it
Fake API responses in tests	**Mock Service Worker (MSW)**, which intercepts `fetch` calls in tests and returns your fake data	None. It's a free npm package
Real-world FX reference rates (once a day)	**Frankfurter API**, free central-bank reference rates: `https://api.frankfurter.dev/v2/rates?base=gbp`	None. No API key, no account
Live ticking FX prices	Your own **price simulator**, a small random walk around the Frankfurter rate, sent over WebSocket or Server-Sent Events	None. You write it
Card numbers	Published test card numbers (e.g. 4242 4242 4242 4242) that payment companies release for testing	None
Optional extra: "connect a bank" demo	**TrueLayer** or **Plaid** sandbox: mock UK banks with fake logins and fake transactions	A free developer sign-up only, no FCA approval for the sandbox. **Never switch to live mode.**
Ask Sterl (AI)	An LLM API (e.g. Anthropic or OpenAI) reading only your fake transactions	A developer account and API key (pay-as-you-go). No regulator involved. Optional: a rule-based version needs nothing

Step-by-step: getting the fake data into Sterl

Step 1: Install the tools

```
npm install @faker-js/faker
npm install -D msw vitest
```

Step 2: Write a seeded generator in `src/mocks/generate.ts` . A seed means "the same random numbers every time", so your tests and screenshots never change by surprise. Every amount is

whole pence.

```
import { faker } from "@faker-js/faker";
faker.seed(42); // same fake data every run

export function makeTransaction(accountId: string) {
  return {
    id: faker.string.uuid(),
    accountId,
    merchant: faker.company.name(),
    category: faker.helpers.arrayElement(["Food", "Transport", "Bills", "Shopping"]),
    amountPence: faker.number.int({ min: 50, max: 15000 }), // £0.50 to £150.00
    date: faker.date.recent({ days: 90 }).toISOString(),
  };
}
```

Step 3: Save it as fixture files. Add a script (`npm run seed`) that writes `accounts.json` , `transactions.json` and `payees.json` into `src/mocks/data/` . Commit these so anyone who clones the repo sees the same bank.

Step 4: Serve it like a real bank API. Create route handlers such as `app/api/accounts/route.ts` and `app/api/transactions/route.ts` that return the JSON. Add a small random delay and an occasional error (e.g. 1 in 20 requests) so you practise loading, error and retry states. Your screens call these with TanStack Query exactly as they would call a real backend.

Step 5: Mock the same API in tests with MSW. Write MSW handlers for the same URLs, so component tests get predictable data without a running server.

Step 6: Fetch real reference FX rates (no key). On the server, call Frankfurter once a day and cache it:

```
// app/api/fx/reference/route.ts
export const revalidate = 86400; // refresh once a day
export async function GET() {
  const res = await fetch("https://api.frankfurter.dev/v2/rates?base=gbp");
  if (!res.ok) return Response.json({ error: "FX source unavailable" }, { status: 502 });
  return Response.json(await res.json());
}
```

Keep a saved copy of one response in `src/mocks/data/fx-fallback.json` so the FX desk still works if the service is down. That's resiliency, which you can talk about in interviews.

Step 7: Make prices tick with a simulator. Start from each reference rate, then several times a second nudge it by a tiny random amount (e.g. ±0.02%), and work out a bid and ask price around it. Stream the ticks to the browser over WebSocket or Server-Sent Events, and batch them into AG Grid. Label the panel clearly: "Simulated prices, not for trading".

Step 8 (optional): Add a bank-connection demo with a sandbox. Sign up free for the TrueLayer or Plaid developer sandbox, connect to their mock bank with the test login they publish, and show the imported fake transactions. Keep the API keys in server-side environment variables (`.env.local` , listed in `.gitignore`) and never request live access.

Launch rules for a demo bank

- Show a clear banner on every page: **"Sterl is a portfolio demo. Not a real bank. All data is fake."**
- Never ask visitors for real names, card numbers, bank logins or money. Use demo logins you create.
- Never commit API keys to GitHub. Use environment variables, and set them in Vercel's settings.
- In the README, explain what would change for a real launch (FCA authorisation, a real ledger, real Confirmation of Payee). Knowing the gap is part of thinking like a bank.

Build plan

Build Sterl during the 2 project hours of each sprint day. Weeks 1–2 build the frontend on fake data. Each later phase adds a realistic layer underneath. The data stays fake throughout.

Sprint weeks 1–2: the frontend (fake data)

Days	What you build in Sterl	Skill practised
1–3	Money helpers in integer pence (round-up, totals, formatting) with tests. Seeded Faker.js generator: accounts, 500 transactions, payees (Steps 1–3 above)	Closures, async/await, modules
4	Zustand store: selected account, card frozen, theme, accessibility mode	Zustand
5	Mock API route handlers with delays and errors (Step 4). Load accounts and transactions with TanStack Query; optimistic card freeze	TanStack Query
6	FX desk watchlist and mock orders in Redux Toolkit; reference rates from Frankfurter with fallback file (Step 6)	Redux Toolkit
7	Rest and review: catch up, refactor, write down what you learned	Reflection
8	App Router pages: dashboard, accounts/[id], transactions, send money, cards, insights, markets	Routing, layouts, loading/error files
9	Account summaries as Server Components; interactive parts as Client Components	Server vs Client Components
10	Transfers and round-up toggles through server actions	Server actions
11	Static help pages, streamed insights page with Suspense	Rendering modes
12	Send money and add payee forms with React Hook Form + Zod, including the simulated payee name check	Forms + validation

Days	What you build in Sterl	Skill practised
13	Polish the dashboard; add the demo banner; first deploy to Vercel	Shipping

Sprint weeks 3–4: testing, performance, job hunt

- Vitest tests for all money maths (round-ups, totals, currency conversion)
- MSW handlers so component tests run on fake API data (Step 5)
- React Testing Library tests for the send money form; Playwright test for the full send money flow
- Live FX desk: the price simulator (Step 7) streaming into AG Grid; keep it smooth with batched updates
- Lighthouse 90+ and a full accessibility audit
- README with screenshots, architecture notes, test badge, "bank thinking" section and a "Data and regulation" section

After the sprint: make it work like a real bank (still fake money)

Phase	What you add	Skills it proves
Backend	Java + Spring Boot (or Node + NestJS) API; PostgreSQL with a double-entry ledger seeded from your Faker data; idempotency keys for payments	A second language, SQL, financial data design
Security	Login with two-factor authentication for demo users, role-based access, audit log table	Auth, security, auditability
Platform	Docker, GitHub Actions CI/CD, cloud deploy, error tracking and logs	CI/CD, observability
AI	Ask Sterl: answers questions from your fake transactions with an LLM	Responsible AI
Fraud	Rules engine for the fraud alert simulator, sending events to the UI in real time	Event-driven architecture
Open banking demo (optional)	TrueLayer or Plaid sandbox connection to a mock bank (Step 8)	Third-party API integration, OAuth-style consent flows

Java + Spring Boot is the better backend choice if JPMorgan is your target, since banks use it widely and the postings value a second language.

Getting hired at JPMorgan

Candidate reports describe the same path for most technology roles: a timed coding test, a recorded video interview, then a final "Super Day". The exact steps vary by programme and

region.

Stage	What happens	How to prepare
1. Application	CV through JPMorgan careers; graduate programme or experienced role	Lead with Sterl: live link, GitHub, 2–3 results with numbers
2. HackerRank test	Timed coding problems, usually easy to medium (sliding window and string/array problems are reported)	LeetCode easy/medium daily; practise under a timer
3. HireVue video	Recorded answers, sometimes explaining your code approach	Practise explaining solutions out loud, on camera
4. Super Day	Back-to-back technical and behavioural interviews	Sterl deep-dive, React questions, live coding

To give yourself the best chance:

- Look for JPMorgan's graduate Software Engineer Program and Code for Good hackathon if you have under 3 years' experience
- Learn the basics of how trading works: orders, FX pairs, bid/ask prices, the trade lifecycle
- Prepare a 5-minute Sterl walkthrough covering one hard decision: real-time performance, money accuracy, security, or why you used fake data and what a real launch would need
- Apply to other UK banks and fintechs in parallel (Barclays, HSBC, Lloyds, Monzo, Starling, Revolut) since one application is a long shot

Sources

- JPMorgan: Software Engineer – JavaScript / React Web UI, London
- JPMorgan: Software Engineer III (UI), DMET Execute, London
- JPMorgan: Sr Lead Software Engineer – UI Developer – React, London
- JPMorgan: Software Engineer III – Expert UI Developer (React JS)
- Chase UK: Round-up account
- Chase UK app on the App Store
- Money to the Masses: Chase UK review
- TechPrep: JPMorgan's interview process (2026)
- Glassdoor: J.P. Morgan software engineer interviews
- Frankfurter: free exchange rates API, no key required
- Faker.js: finance module
- Finexer: Open banking sandboxes in the UK compared
- TrueLayer: Test payments in sandbox